



网络信息处理实验报告

题目 实验 2：捕获并分析网络分组

姓名：马承乾

学号：22920212204180

院系：人工智能系

专业：人工智能

年级：2021 级

指导教师：陈毅东

2024 年 4 月 3 日

一、 目录

一、	实验名称	4
二、	实验任务	4
1.	学习 Wireshark 的过滤器使用	4
2.	捕获与分析 ARP 报文	4
3.	捕获与分析 ICMP 报文	4
4.	捕获与分析 IP 报文	4
三、	实验环境及工具	4
四、	实验记录与结果分析	4
1.	学习 Wireshark 的过滤器使用	4
1)	捕获过滤器 (Capture Filter)	4
2)	显示过滤器 (Display Filter)	6
3)	对运算符的使用	8
2.	捕获与分析 ARP 报文	9
1)	帧	10
2)	Ethernet (以太网):	11
3)	Address Resolution Protocol (ARP) (请求):	11
3.	捕获与分析 ICMP 报文	11
1)	第一层 frame:	13
2)	第二层数据链路层	13
3)	internet protocol version 4:	14
4)	第四层传输层	14
4.	捕获与分析 IP 报文	14
1)	Frame (帧):	15
2)	Ethernet II (以太网 II):	16
3)	Internet Protocol Version 4 (IPv4) (互联网协议第四版):	16
4)	Transmission Control Protocol (TCP) (传输控制协议):	17
5)	Transport Layer Security (TLS) (传输层安全):	18

五、	遇到的问题与解决.....	18
----	---------------	----

一、 实验名称

捕获并分析网络分组。

二、 实验任务

1. 学习 Wireshark 的过滤器使用
2. 捕获与分析 ARP 报文
3. 捕获与分析 ICMP 报文
4. 捕获与分析 IP 报文

三、 实验环境及工具

本实验使用 64 位 Windows 系统下的 Wireshark4.2.4 版本进行实验。

四、 实验记录与结果分析

1. 学习 Wireshark 的过滤器使用

Wireshark 的过滤器分为两种：捕获过滤器和显示过滤器。它们各自有不同的用途和应用时机，但都是为了帮助用户更有效地分析网络流量。

1) 捕获过滤器（Capture Filter）

捕获过滤器在数据包被捕获之前应用，用于减少捕获的数据量。这样可以节省资源，提高效率，尤其是在高流量网络中。捕获过滤器格式如下：

语法：

Protocol	Direction	Host(s)	Value	Logical Operations	Other expression
----------	-----------	---------	-------	--------------------	------------------

例子：

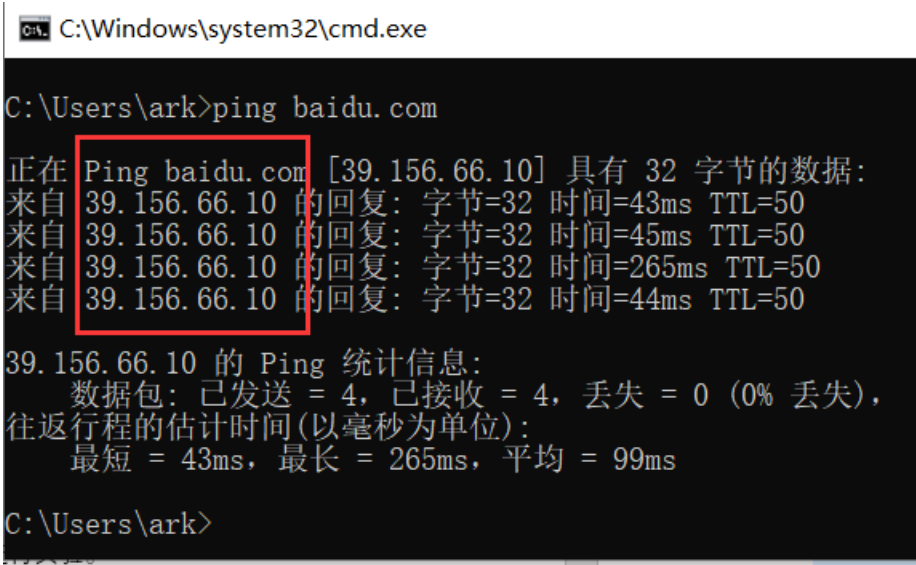
tcp	dst	10.1.1.1	80	and	tcp dst 10.2.2.2 3128
-----	-----	----------	----	-----	-----------------------

使用方法：

- 打开 Wireshark。
- 选择“捕获”。
- 选择捕获过滤器。
- 开始捕获，Wireshark 将只捕获符合过滤条件的数据包。

示例：

使用 ping 命令获得来自百度的响应：



图表 1 使用 ping 命令与百度交互

获得此响应后，观察到该数据包长度为 74：

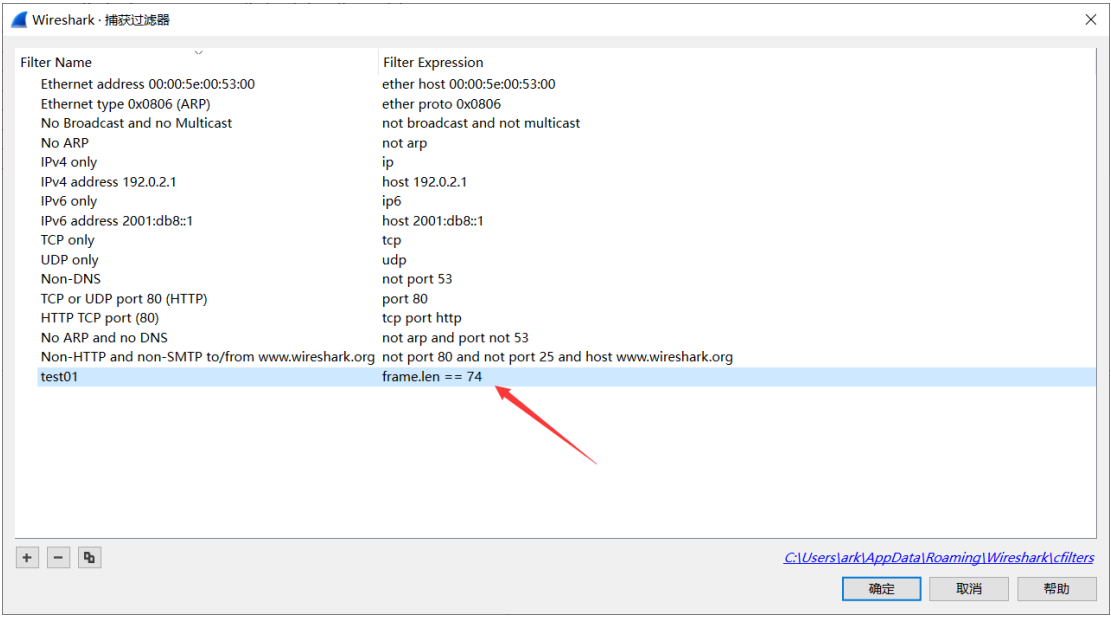
No.	Time	Source	Destination	Protocol	Length	Info
46	12.187845	192.168.2.157	120.241.130.179	SSL	111	Continu
47	12.221260	120.241.130.179	192.168.2.157	TCP	60	443 → 2
48	13.866454	192.168.2.157	192.168.2.1	DNS	69	Standar
49	13.898614	192.168.2.1	192.168.2.157	DNS	101	Standar
50	13.903764	192.168.2.157	39.156.66.10	ICMP	74	Echo (p
51	13.947295	39.156.66.10	192.168.2.157	ICMP	74	Echo (p
52	14.925572	192.168.2.157	39.156.66.10	ICMP	74	Echo (p
53	14.929410	192.168.2.157	192.168.2.1	DNS	75	Standar
54	14.957998	192.168.2.1	192.168.2.157	DNS	368	Standar
55	14.959858	192.168.2.157	112.5.63.116	TCP	66	2697 →
56	14.970322	39.156.66.10	192.168.2.157	ICMP	74	Echo (p
57	14.970499	112.5.63.116	192.168.2.157	TCP	66	443 → 2
58	14.970680	192.168.2.157	112.5.63.116	TCP	54	2697 →
59	14.972166	192.168.2.157	112.5.63.116	TLSv1.3	371	Client
60	14.981233	112.5.63.116	192.168.2.157	TCP	54	443 → 2
61	14.986793	112.5.63.116	192.168.2.157	TLSv1.3	1494	Server
62	14.986793	112.5.63.116	192.168.2.157	TCP	1494	443 → 2
63	14.986793	112.5.63.116	192.168.2.157	TCP	1270	443 → 2

- `frame.len == 74`：只捕获目标端口为 80 的 TCP 数据包。



图表 2：使用捕获过滤器

选择捕获-->选项;



图表 3：设置捕获过滤器

The image shows a Wireshark packet capture results window. The 'Length' column is highlighted in red. The table below shows the captured packets:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.2.157	39.156.66.10	ICMP	74	Echo (ping)
2	0.053370	39.156.66.10	192.168.2.157	ICMP	74	Echo (ping)
3	1.013270	192.168.2.157	39.156.66.10	ICMP	74	Echo (ping)
4	1.063965	39.156.66.10	192.168.2.157	ICMP	74	Echo (ping)
5	1.577620	192.168.2.157	120.253.253.33	TCP	55	2733 → 443 来自 39.156.66.10 的回复: 字节=32 时间=53ms TTL=50
6	1.620617	120.253.253.33	192.168.2.157	TCP	66	443 → 2733 来自 39.156.66.10 的回复: 字节=32 时间=50ms TTL=50
7	2.044613	192.168.2.157	39.156.66.10	ICMP	74	Echo (ping)
8	2.092202	39.156.66.10	192.168.2.157	ICMP	74	Echo (ping)
9	2.624506	120.241.130.179	192.168.2.157	SSL	647	Continuation Data
10	2.624897	192.168.2.157	120.241.130.179	SSL	111	Continuation Data
11	2.654526	120.241.130.179	192.168.2.157	TCP	54	443 → 2579 来自 39.156.66.10 的回复: 字节=32 时间=47ms TTL=50
12	3.058213	192.168.2.157	39.156.66.10	ICMP	74	Echo (ping)
13	3.105866	39.156.66.10	192.168.2.157	ICMP	74	Echo (ping)
14	5.081879	zte_79:6c:cc	Intel_21:eb:52	ARP	42	Who has 192.168.2.157? is at 9c:12:97:76:21:eb:52
15	5.081909	Intel_21:eb:52	zte_79:6c:cc	ARP	42	192.168.2.157 is at 9c:12:97:76:21:eb:52
16	5.594378	120.241.130.179	192.168.2.157	SSL	391	Continuation Data
17	5.597409	192.168.2.157	120.241.130.179	SSL	111	Continuation Data
18	5.627577	120.241.130.179	192.168.2.157	TCP	54	443 → 2579 [ACK] Seq=931 Ack=115 Win=30855 Len=0

图表 4：捕获结果

过滤结果显示成功捕获到了长度为 74 的数据包，此处之所以还有其他长度不为 74 的数据包存在，原因是捕获过滤器中还有别的捕获规则。

2) 显示过滤器（Display Filter）

显示过滤器在数据包被捕获之后应用，用于在 Wireshark 的界面中显示符合特定条件的数据包。这不会改变已捕获的数据包，只是临时隐藏不符合条件的数据包。显示过滤器格式如下：

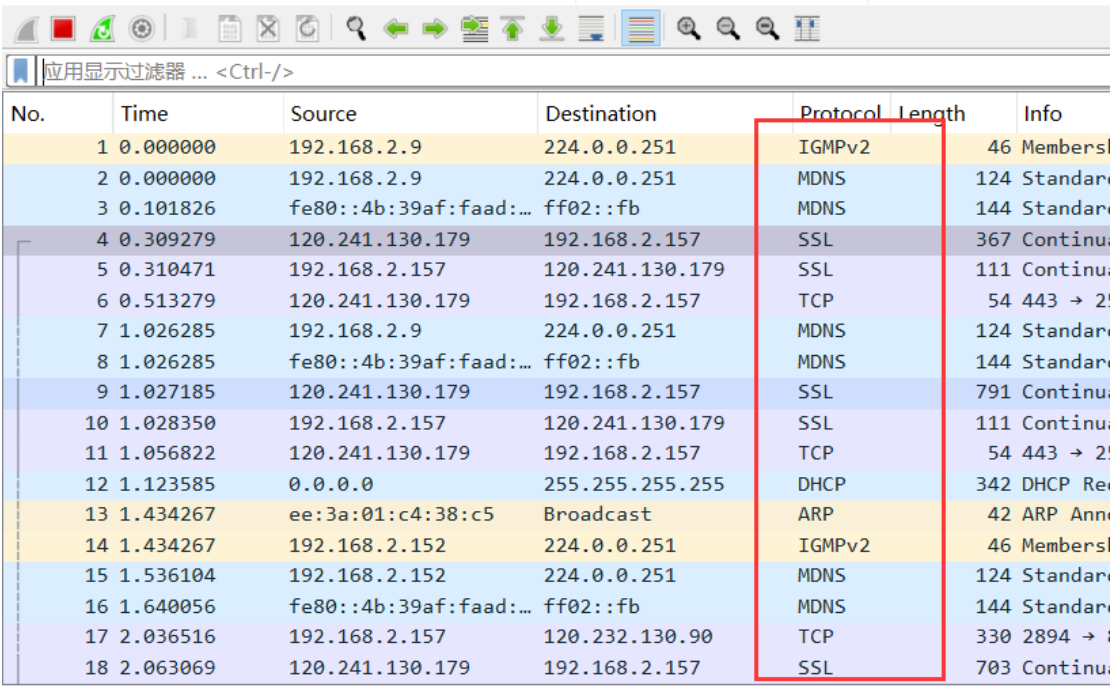
语法：	Protocol	.	String 1	.	String 2	Comparison operator	Value	Logical Operations	Other expression
例子：	ftp	.	passive	.	ip	==	10.2.3.4	xor	icmp.type

图表 5 显示过滤器格式

使用方法：

1. 在 Wireshark 的主界面顶部，找到“显示过滤器”栏。
2. 输入过滤表达式。
3. 按下 Enter 键或点击过滤器栏旁边的“应用”按钮。

示例：



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.2.9	224.0.0.251	IGMPv2	46	Membersl
2	0.000000	192.168.2.9	224.0.0.251	MDNS	124	Standar
3	0.101826	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	Standar
4	0.309279	120.241.130.179	192.168.2.157	SSL	367	Continu
5	0.310471	192.168.2.157	120.241.130.179	SSL	111	Continu
6	0.513279	120.241.130.179	192.168.2.157	TCP	54	443 → 2
7	1.026285	192.168.2.9	224.0.0.251	MDNS	124	Standar
8	1.026285	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	Standar
9	1.027185	120.241.130.179	192.168.2.157	SSL	791	Continu
10	1.028350	192.168.2.157	120.241.130.179	SSL	111	Continu
11	1.056822	120.241.130.179	192.168.2.157	TCP	54	443 → 2
12	1.123585	0.0.0.0	255.255.255.255	DHCP	342	DHCP Re
13	1.434267	ee:3a:01:c4:38:c5	Broadcast	ARP	42	ARP Ann
14	1.434267	192.168.2.152	224.0.0.251	IGMPv2	46	Membersl
15	1.536104	192.168.2.152	224.0.0.251	MDNS	124	Standar
16	1.640056	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	Standar
17	2.036516	192.168.2.157	120.232.130.90	TCP	330	2894 →
18	2.063069	120.241.130.179	192.168.2.157	SSL	703	Continu

图表 6 未过滤前

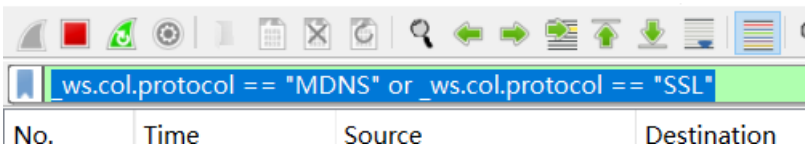
设置过滤条件为：

`_ws.col.protocol == "MDNS" or _ws.col.protocol == "SSL"`

_ws.col.protocol 是一个显示过滤器（display filter），它用于指定数据包的协议。这个过滤器表达式的意思是：

- 显示所有协议为 MDNS（多播 DNS）的数据包。
- 或者显示所有协议为 SSL（安全套接层）的数据包。

逻辑运算符 or 允许同时满足两个条件中的任何一个，从而在 Wireshark 中显示这两类协议的数据包。



No.	Time	Source	Destination
-----	------	--------	-------------

图表 7 输入显示过滤器

_ws.col.protocol == "MDNS" or _ws.col.protocol == "SSL"						
No.	Time	Source	Destination	Protocol	Length	Ir
2	0.000000	192.168.2.9	224.0.0.251	MDNS	124	S
3	0.101826	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	S
4	0.309279	120.241.130.179	192.168.2.157	SSL	367	C
5	0.310471	192.168.2.157	120.241.130.179	SSL	111	C
7	1.026285	192.168.2.9	224.0.0.251	MDNS	124	S
8	1.026285	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	S
9	1.027185	120.241.130.179	192.168.2.157	SSL	791	C
10	1.028350	192.168.2.157	120.241.130.179	SSL	111	C
15	1.536104	192.168.2.152	224.0.0.251	MDNS	124	S
16	1.640056	fe80::4b:39af:faad:...	ff02::fb	MDNS	144	S
18	2.063069	120.241.130.179	192.168.2.157	SSL	703	C
19	2.065476	192.168.2.157	120.241.130.179	SSL	111	C
25	2.119475	192.168.2.157	120.241.130.179	SSL	207	C
27	2.151010	192.168.2.157	120.241.130.179	SSL	103	C
29	2.174783	120.241.130.179	192.168.2.157	SSL	735	C
31	2.201473	192.168.2.157	120.241.130.179	SSL	711	C
32	2.231009	120.241.130.179	192.168.2.157	SSL	127	C
33	2.231079	192.168.2.157	120.241.130.179	SSL	207	C

图表 8 过滤后

观察过滤结果，果然只剩下满足显示过滤器条件的结果留了下来。

3) 对运算符的使用

可使用的运算符如下：

英文写法：	C语言写法：	含义：
and	&&	逻辑与
or		逻辑或
xor	^^	逻辑异或
not	!	逻辑非

图表 9：逻辑运算符

英文写法：	C语言写法：	含义：
eq	==	等于
ne	!=	不等于
gt	>	大于
lt	<	小于
ge	>=	大于等于
le	<=	小于等于

图表 10：比较运算符

逻辑运算符有着另一种使用方法：

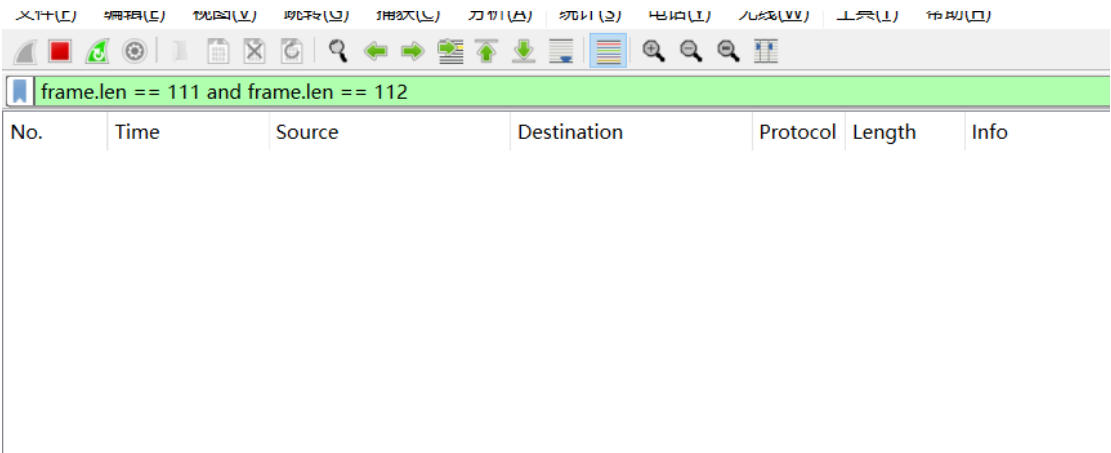
使用`&&`表示逻辑与（AND），`||`表示逻辑或（OR），`!`表示逻辑非（NOT）。

对 OR 的使用可见图表 8 的过滤结果。

此处举例说明一个不规范的用法：

frame.len == 111 and frame.len == 112

这条显示过滤器希望筛选出长度同时是 111 和 112 的数据包，但一个数据包的长度只能是一个准确的数字，故不可能筛选出结果：

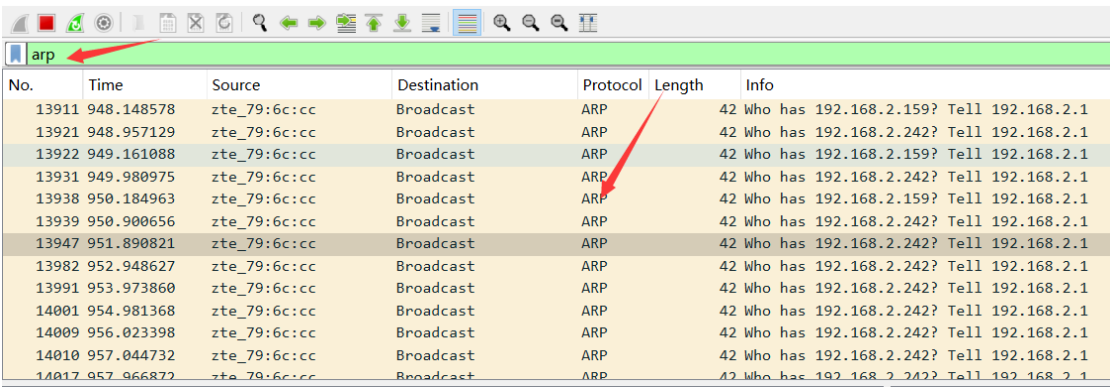


图表 11：显示过滤器结果

观察结果，果然空空如也。

通过合理使用 Wireshark 的过滤器，我们可以更快速地定位问题，分析网络行为，以及提取有价值的信息。

2. 捕获与分析 ARP 报文

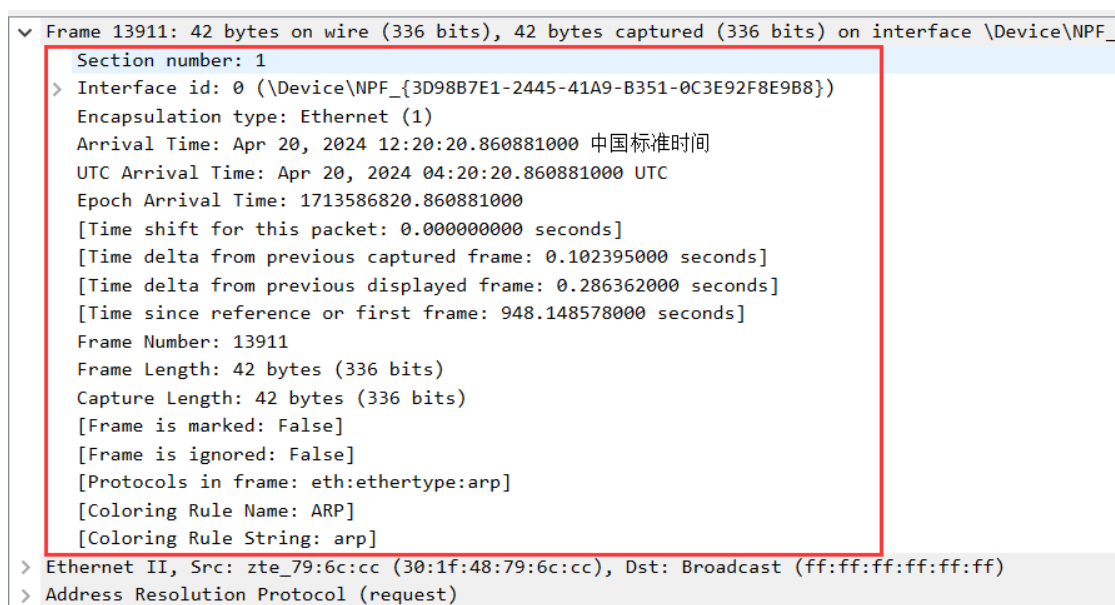


图表 12 捕获 arp 报文

如图，已经捕获到大量 arp 报文。ARP（地址解析协议，Address Resolution Protocol）是一种用于网络层和数据链路层之间的通信协议，它用于将网络层的 IP 地址解析为数据链路层的 MAC 地址。其格式如下：



图表 13 ARP 报文格式

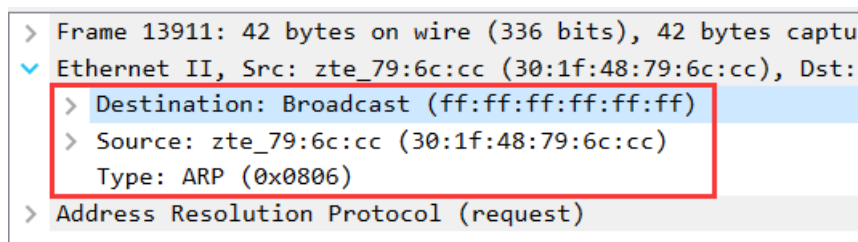


图表 14：第一层

1) 帧

在数据链路层，所有的数据传输都是以帧为单位进行的。帧是数据链路层的打包数据的基本单位，它包括了控制信息和数据本身。

在以太网中，帧通常指的是以太网帧 (Ethernet Frame)，它包含了目的 MAC 地址、源 MAC 地址、类型字段 (指示上层协议类型) 和数据载荷。



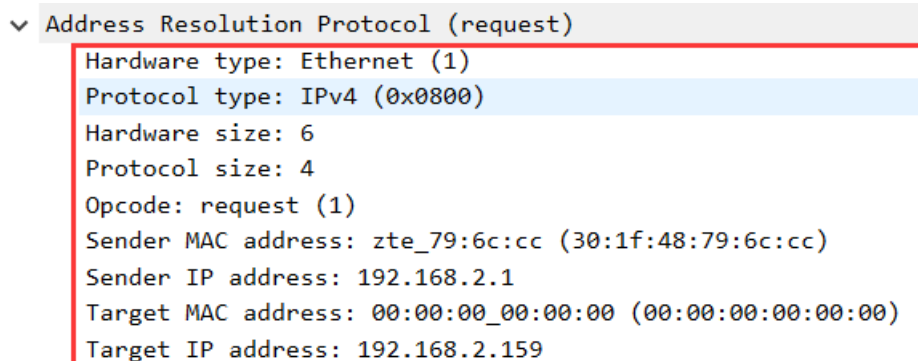
图表 15 第二层

2) Ethernet (以太网):

以太网是局域网（LAN）通信的一种标准，定义了 OSI 模型的物理层和数据链路层的规范。

在以太网帧中，以太网头包含了目的 MAC 地址和源 MAC 地址，这两个地址分别标识了帧的接收者和发送者。

类型字段指出了帧中数据载荷的协议类型，使得接收设备能够知道如何处理这个帧的数据部分。



```
▼ Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: zte_79:6c:cc (30:1f:48:79:6c:cc)
  Sender IP address: 192.168.2.1
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 192.168.2.159
```

图表 16 第三层

3) Address Resolution Protocol (ARP) (请求):

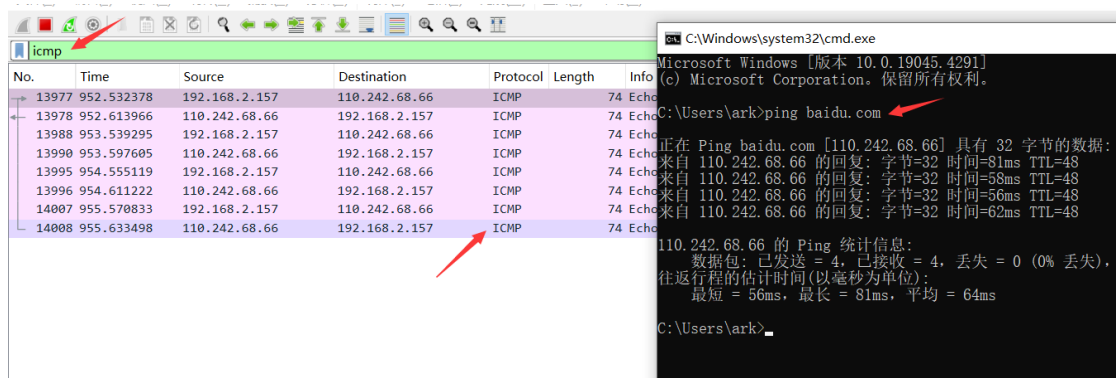
ARP 是一种网络层和数据链路层之间的桥梁协议，用于将 IP 地址映射到 MAC 地址。

当一个设备需要将一个数据包发送给局域网内的另一个设备，但只知道对方的 IP 地址时，它会在局域网内广播一个 ARP 请求，询问拥有目标 IP 地址的设备的 MAC 地址。

ARP 请求报文包含发送者的 IP 地址和 MAC 地址，以及目标 IP 地址和未指定的 MAC 地址（通常为全零的 MAC 地址，表示广播）。

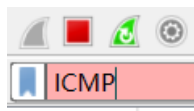
3. 捕获与分析 ICMP 报文

考虑到使用捕获过滤器进行捕获，捕获过滤器中初始化了许多过滤条件，这导致捕获结果中包含许多不满足我们设置的其他内容，且在使用过后需要将其删除，比较麻烦。方便起见，使用显示过滤器：



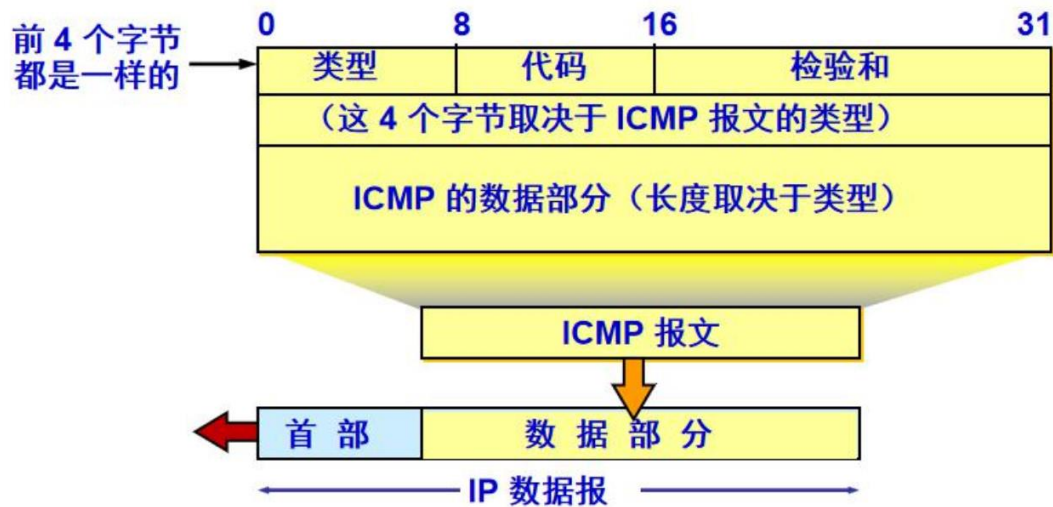
图表 17: 捕获到的 icmp 报文

注意，只能使用小写字母，使用大写字母则无法正常捕获：



图表 18 捕获被报错

ICMP 报文格式如下：



图表 19: ICMP 报文格式

接下来我们分析 ICMP 报文的内容：

```

Frame 13977: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{...}
  Section number: 1
  > Interface id: 0 (\Device\NPF_{3D98B7E1-2445-41A9-B351-0C3E92F8E9B8})
  Encapsulation type: Ethernet (1)
  Arrival Time: Apr 20, 2024 12:20:25.244681000 中国标准时间
  UTC Arrival Time: Apr 20, 2024 04:20:25.244681000 UTC
  Epoch Arrival Time: 1713586825.244681000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.005502000 seconds]
  [Time delta from previous displayed frame: 0.000000000 seconds]
  [Time since reference or first frame: 952.532378000 seconds]
  Frame Number: 13977
  Frame Length: 74 bytes (592 bits)
  Capture Length: 74 bytes (592 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:icmp:data]
  [Coloring Rule Name: ICMP]
  [Coloring Rule String: icmp || icmpv6]
  > Ethernet II, Src: Intel_21:eb:52 (9c:29:76:21:eb:52), Dst: zte_79:6c:cc (30:1f:48:79:6c:cc)
  > Internet Protocol Version 4, Src: 192.168.2.157, Dst: 110.242.68.66
  > Internet Control Message Protocol

```

图表 20：第一层 frame 数据

1) 第一层 frame:

这是数据包的最外层，表示一个以太网帧（Ethernet II）。以太网帧是网络通信的基本单位，包含了源 MAC 地址、目的 MAC 地址、类型字段（指示上层协议类型）以及数据载荷。

```

> Frame 13977: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{...}
  > Ethernet II, Src: Intel_21:eb:52 (9c:29:76:21:eb:52), Dst: zte_79:6c:cc (30:1f:48:79:6c:cc)
    > Destination: zte_79:6c:cc (30:1f:48:79:6c:cc)
    > Source: Intel_21:eb:52 (9c:29:76:21:eb:52)
    Type: IPv4 (0x0800)
  > Internet Protocol Version 4, Src: 192.168.2.157, Dst: 110.242.68.66
  > Internet Control Message Protocol

```

图表 21：第二层数据链路层

2) 第二层数据链路层

ethernet II：这表示数据包的第二层（数据链路层），遵循 IEEE 802.3 标准。在这一层，可以看到数据包的源 MAC 地址和目的 MAC 地址，以及用于错误检测的帧校验序列（FCS）。

```
> Frame 13977: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{...}
> Ethernet II, Src: Intel_21:eb:52 (9c:29:76:21:eb:52), Dst: zte_79:6c:cc (30:1f:48:79:6c:cc)
✓ Internet Protocol Version 4, Src: 192.168.2.157, Dst: 110.242.68.66
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 60
    Identification: 0x819c (33180)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: ICMP (1)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.2.157
    Destination Address: 110.242.68.66
> Internet Control Message Protocol
```

图表 22：第三层网络层

3) internet protocol version 4:

这表示数据包的第三层（网络层），即 IPv4。在这一层，您可以看到源 IP 地址、目的 IP 地址、生存时间（TTL）、协议类型（这里是 ICMP），以及其他一些控制信息。

```
✓ Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x4d2d [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 46 (0x002e)
  Sequence Number (LE): 11776 (0x2e00)
  [Response frame: 13978]
  > Data (32 bytes)
```

图表 23：第四层传输层

4) 第四层传输层

这表示数据包的第四层（传输层），即 ICMP。ICMP 是一种网络层协议，用于在 IP 主机、路由器之间传递控制消息。ICMP 消息通常封装在 IP 数据包内，用于报告错误情况（如目的地不可达）或进行诊断（如 ping 操作）。

4. 捕获与分析 IP 报文

一个典型的 IP 报文在网络中传输时，会经过多个层次的封装，每个层次负责不同的功能。我们捕获的 IP 报文如下：

No.	Time	Source	Destination	Protocol	Length	Info
13904	948.040393	20.44.248.140	192.168.2.157	TCP	1494	443 → 3133 [ACK] Seq=144
13905	948.040393	20.44.248.140	192.168.2.157	TCP	1494	443 → 3133 [ACK] Seq=144
13906	948.040393	20.44.248.140	192.168.2.157	TCP	1031	[TCP Previous segment no
13907	948.040393	20.44.248.140	192.168.2.157	TCP	1494	[TCP Out-Of-Order] 443 →
13908	948.040469	192.168.2.157	20.44.248.140	TCP	66	3133 → 443 [ACK] Seq=203
13909	948.040520	192.168.2.157	20.44.248.140	TCP	54	3133 → 443 [ACK] Seq=203
13910	948.046183	192.168.2.157	20.44.248.140	TLSv1.2	212	Client Key Exchange, Cha
13912	948.238664	20.44.248.140	192.168.2.157	TLSv1.2	105	Change Cipher Spec, Encr
13913	948.238738	192.168.2.157	20.44.248.140	TCP	54	3133 → 443 [ACK] Seq=361
13914	948.244360	192.168.2.157	20.44.248.140	TLSv1.2	522	Application Data
13915	948.244558	192.168.2.157	20.44.248.140	TLSv1.2	1253	Application Data
13916	948.445755	20.44.248.140	192.168.2.157	TCP	54	443 → 3133 [ACK] Seq=534
13917	948.446571	20.44.248.140	192.168.2.157	TLSv1.2	851	Application Data

图表 24 捕获 Ip 报文

IP 报文格式如下：

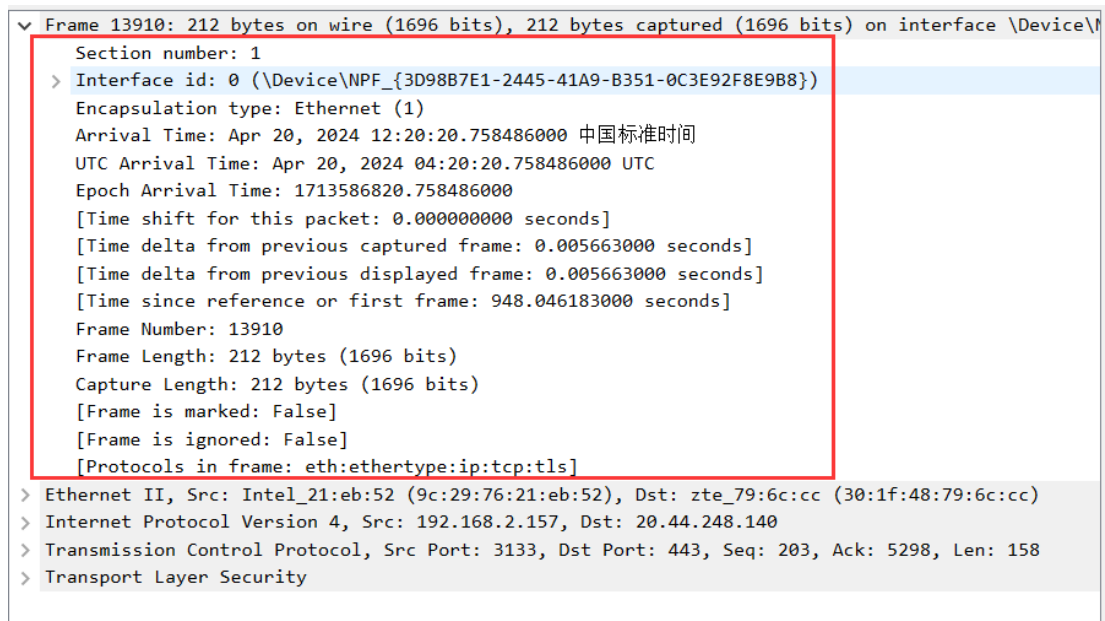


图表 25: IP 报文格式

1) Frame (帧):

帧是数据链路层的封装单位，对于以太网，这通常是以太网帧（Ethernet II 帧）。

帧包含头部和尾部，其中头部包括目的 MAC 地址、源 MAC 地址和类型字段。类型字段用于指定上层协议类型。

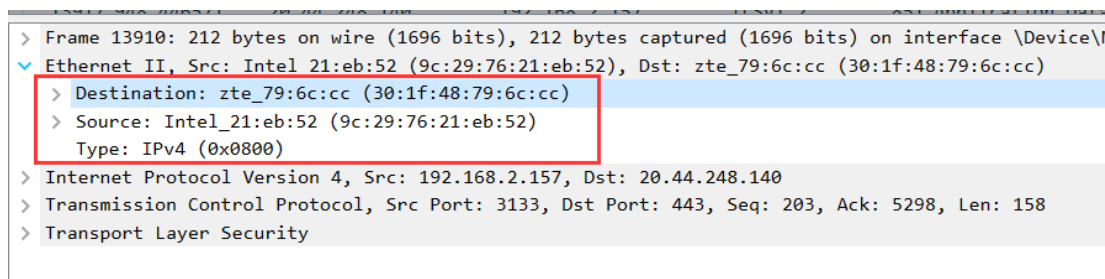


图表 26 第一层

2) Ethernet II (以太网 II):

以太网 II 帧是最常见的以太网帧格式，它包含目的 MAC（6 字节）、源 MAC（6 字节）和类型/长度（2 字节）字段，后跟数据载荷和帧校验序列（FCS，4 字节）。

以太网 II 帧负责在局域网内从一个设备传输到另一个设备。



图表 27：第二层

3) Internet Protocol Version 4 (IPv4) (互联网协议第四版):

IPv4 负责在网络层提供源和目的 IP 地址，处理数据包从源到目的地的传递。

IPv4 头部包含版本、头部长度的总长度、标识、标志、片偏移、生存时间（TTL）、协议、头部校验和、源 IP 地址、目的 IP 地址等字段。


```
13917 948 146571 20 44 248 140 192 168 2 157 TLSv1 2 851 Application Data
> Frame 13910: 212 bytes on wire (1696 bits), 212 bytes captured (1696 bits) on interface \Device\
> Ethernet II, Src: Intel_21:eb:52 (9c:29:76:21:eb:52), Dst: zte_79:6c:cc (30:1f:48:79:6c:cc)
✓ Internet Protocol Version 4, Src: 192.168.2.157, Dst: 20.44.248.140
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        Total Length: 198
        Identification: 0x2514 (9492)
    > 010. .... = Flags: 0x2, Don't fragment
        ...0 0000 0000 0000 = Fragment Offset: 0
        Time to Live: 128
        Protocol: TCP (6)
        Header Checksum: 0x0000 [validation disabled]
        [Header checksum status: Unverified]
        Source Address: 192.168.2.157
        Destination Address: 20.44.248.140
> Transmission Control Protocol, Src Port: 3133, Dst Port: 443, Seq: 203, Ack: 5298, Len: 158
> Transport Layer Security
```

图表 28 第三层

4) Transmission Control Protocol (TCP) (传输控制协议):

TCP 是一种传输层协议，负责在 IP 网络中提供可靠的、有序的和错误检测功能的数据传输。

TCP 头部包含源端口、目的端口、序列号、确认号、头部长度的控制标志（如 SYN, ACK, FIN 等）、窗口大小、校验和、紧急指针等字段。

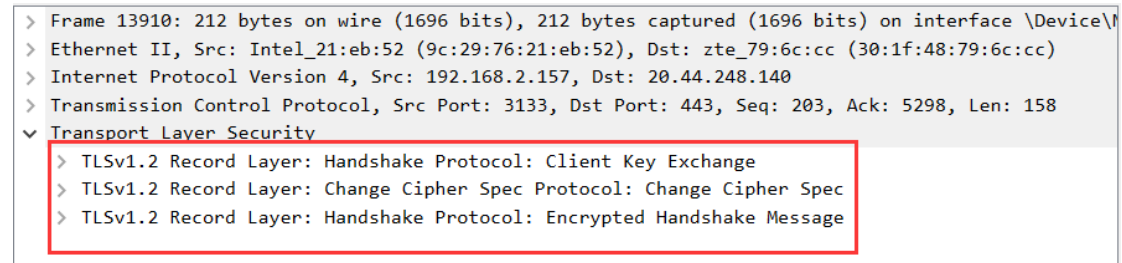
```
> Frame 13910: 212 bytes on wire (1696 bits), 212 bytes captured (1696 bits) on interface \Device\
> Ethernet II, Src: Intel_21:eb:52 (9c:29:76:21:eb:52), Dst: zte_79:6c:cc (30:1f:48:79:6c:cc)
> Internet Protocol Version 4, Src: 192.168.2.157, Dst: 20.44.248.140
✓ Transmission Control Protocol, Src Port: 3133, Dst Port: 443, Seq: 203, Ack: 5298, Len: 158
    Source Port: 3133
    Destination Port: 443
    [Stream index: 211]
    > [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 158]
    Sequence Number: 203 (relative sequence number)
    Sequence Number (raw): 1108577890
    [Next Sequence Number: 361 (relative sequence number)]
    Acknowledgment Number: 5298 (relative ack number)
    Acknowledgment number (raw): 4189800160
    0101 .... = Header Length: 20 bytes (5)
    > Flags: 0x018 (PSH, ACK)
    Window: 1024
    [Calculated window size: 262144]
    [Window size scaling factor: 256]
    Checksum: 0xd0b6 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    > [Timestamps]
    > [SEQ/ACK analysis]
```

图表 29 第四层

5) Transport Layer Security (TLS) (传输层安全):

TLS 是一种安全协议，用于在网络通信过程中提供隐秘性和数据完整性。

它通常在 TCP 连接建立之后，用于加密 TCP 连接上传输的数据，保护数据免受中间人攻击。



图表 30 第五层

五、 遇到的问题与解决

本次实验较为简单，未遇见问题。